

Cycle Analysis in EvalData

A Small Algorithm Note

EvalData Repository Example

April 2026

1 Purpose

This note explains the current cycle-analysis workflow in EvalData. The examples use the built-in spectral demo, which includes a deterministic impact marker and repeated ringing events.

Cycle analysis in EvalData is based on segmentation first and statistics second. The quality of the result depends strongly on whether the cycle boundaries are meaningful.

2 Two Cycle Modes

The current UI exposes two modes:

- **fixed_length**: build cycles as equal row blocks,
- **rising_edge**: detect cycle starts from threshold crossings on a reference signal.

2.1 Fixed-Length Segmentation

For a chosen cycle length L , the data is split into half-open intervals

$$[0, L), [L, 2L), [2L, 3L), \dots$$

until there are not enough rows left to form another full cycle.

2.2 Rising-Edge Segmentation

For reference signal $r[n]$ and threshold τ , EvalData first marks samples above threshold:

$$b[n] = \mathbf{1}_{\{r[n] \geq \tau\}}.$$

A rising edge occurs when

$$b[n] = 1 \quad \text{and} \quad b[n-1] = 0.$$

Consecutive rising edges define candidate cycle ranges. Short ranges below the configured minimum length are discarded.

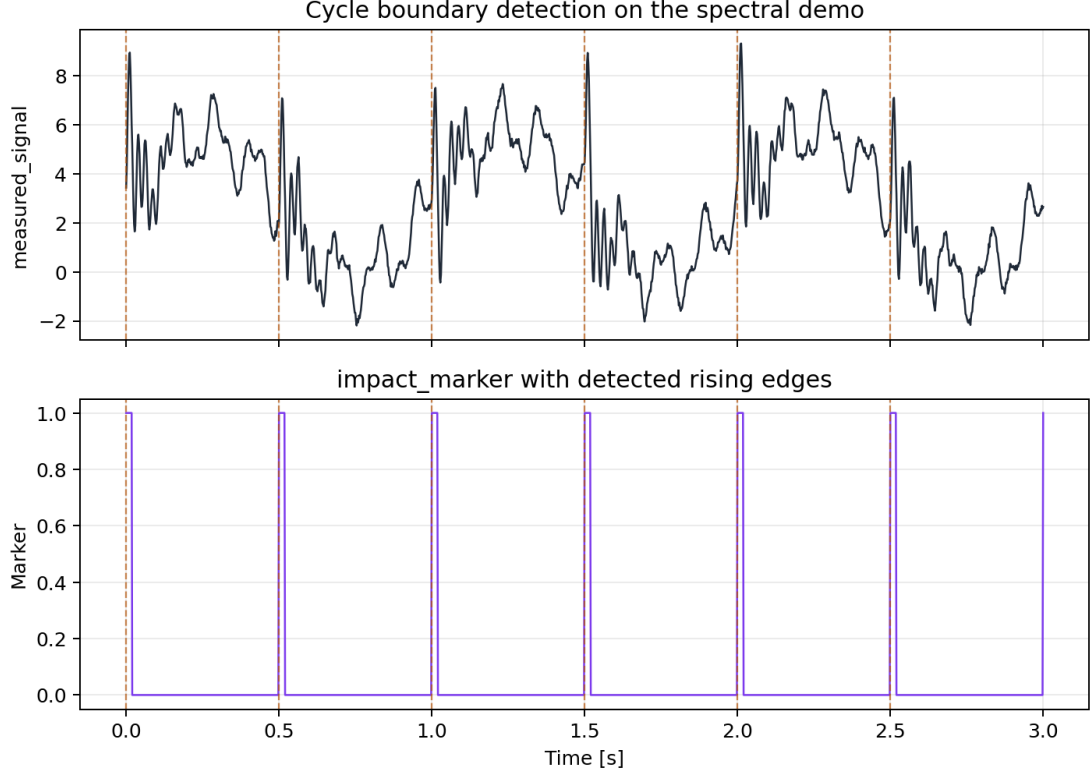


Figure 1: Rising-edge cycle detection on the built-in spectral demo. The marker channel provides explicit threshold crossings for cycle boundary detection.

3 Cycle Alignment

Once valid cycle ranges are collected, EvalData converts each range into a numeric array. Because detected cycles may differ slightly in length, the current implementation aligns them by truncating every cycle to the shortest valid cycle length:

$$L_{\text{align}} = \min_i L_i.$$

This yields a cycle matrix

$$X \in \mathbb{R}^{M \times L_{\text{align}}},$$

where M is the number of detected cycles.

4 Per-Cycle Metrics

For each cycle $x_i[n]$, EvalData reports:

$$\begin{aligned} \mu_i &= \frac{1}{L_i} \sum_{n=0}^{L_i-1} x_i[n], \\ \sigma_i &= \sqrt{\frac{1}{L_i-1} \sum_{n=0}^{L_i-1} (x_i[n] - \mu_i)^2}, \\ \text{RMS}_i &= \sqrt{\frac{1}{L_i} \sum_{n=0}^{L_i-1} x_i[n]^2}, \end{aligned}$$

$$P2P_i = \max_n x_i[n] - \min_n x_i[n].$$

Minimum and maximum values are also stored per cycle.

5 Representative Cycle

After alignment, EvalData computes a representative cycle summary across the cycle matrix column by column:

$$\bar{x}[k] = \frac{1}{M} \sum_{i=1}^M X_{ik},$$

with corresponding standard deviation, minimum, and maximum at each aligned step k .

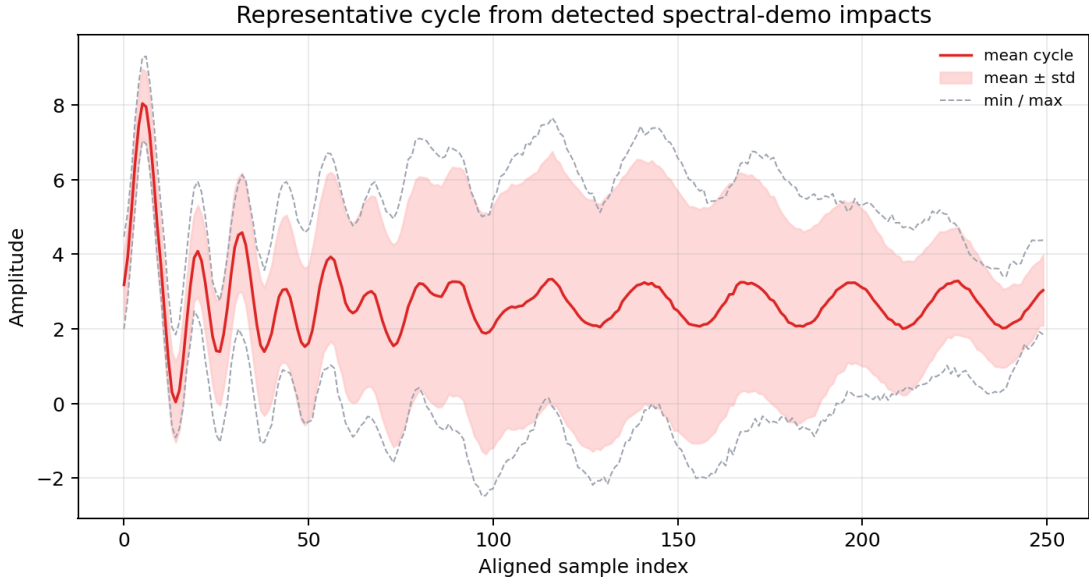


Figure 2: Representative cycle summary from detected spectral-demo impacts. The band shows mean $\pm$ standard deviation across aligned cycles.

6 What the Demo Shows

The spectral demo repeats impacts every 0.5 seconds and adds decaying ringing after each event. That makes it a good control case for both segmentation strategies:

- a fixed length of 250 rows matches 0.5 seconds at 500 Hz,
- the impact marker allows explicit threshold-based cycle detection.

7 Practical Interpretation Limits

- Fixed-length segmentation is only valid if the cycle length is known or approximately constant.
- Rising-edge detection depends on threshold choice and signal cleanliness.

- Truncating all cycles to the shortest length simplifies alignment but discards tail samples from longer cycles.
- Cycle metrics summarize segments; they do not prove physical repeatability by themselves.

8 Conclusion

EvalData's cycle workflow is deliberately simple: detect or define cycle ranges, align them, and summarize them. That is enough to make repeat structure visible, but the mathematical assumptions behind the segmentation step should always be stated explicitly.